50 C Programming Practice Questions

For absolute beginners — arranged easiest to hardest

How to use this sheet: work through the questions in order. Don't skip ahead — each question quietly reuses an idea from the one before it and adds one new thing. Write and actually run every program; don't just think through it in your head. A short hint is given under each question if you get stuck.

Level 1: Getting Started (Q1-8)

1. Write a program that prints the message **Hello, World!** to the screen.

   Hint: Use #include <stdio.h> and printf(). Every C program needs a main() function.

2. Write a program that declares an integer variable, assigns it the value 25, and prints it.

   Hint: Declare with int x = 25; and print with printf("%d", x);

3. Write a program that declares three variables: your age (int), your height in meters (float), and the first letter of your name (char). Print all three on separate lines.

   Hint: Use format specifiers %d for int, %f for float, %c for char.

4. Write a program that stores two integers and prints their sum, difference, product, and quotient.

   Hint: Store the two numbers in variables first, then compute each result before printing.

5. Write a program that takes an integer input from the user using scanf() and prints double that value.

   Hint: scanf("%d", &num); — don't forget the & before the variable name.

6. Write a program that asks the user for two numbers and prints their average.

   Hint: Watch out: dividing two ints gives an int result. Consider making one variable a float.

7. Write a program that swaps the values of two variables using a third temporary variable, and prints the values before and after swapping.

   Hint: temp = a; a = b; b = temp; — print before you swap and after you swap.

8. Write a program that converts a temperature from Celsius to Fahrenheit, given a Celsius value typed in by the user.

   *Hint: Formula: F = (C * 9/5) + 32. Be careful with integer division on 9/5.*

Level 2: Decisions (Q9-15)

9. Write a program that reads one integer and prints whether it is positive, negative, or zero.

   Hint: Use if / else if / else.

10. Write a program that reads an integer and prints whether it is even or odd.

    Hint: Use the modulus operator %. A number is even if number % 2 == 0.

11. Write a program that reads three different numbers and prints the largest of the three.

    Hint: You can do this with nested if statements, or with a chain of else if.

12. Write a program that reads a person's age and prints "Child", "Teenager", or "Adult" based on set age ranges you choose.

    Hint: Decide your own ranges, e.g. under 13, 13-19, 20 and above.

13. Write a program that reads a character and prints whether it is a vowel or a consonant. Assume the input is always a lowercase letter.

    Hint: Compare the char against 'a','e','i','o','u' using || (logical OR).

14. Write a program that reads a year and prints whether it is a leap year.

Hint: A year is a leap year if divisible by 4, but not by 100, unless also divisible by 400.

15. Write a program that reads a number from 1 to 7 and prints the matching day name (1 = Monday, ... 7 = Sunday) using a switch statement.

Hint: Remember to include a default case and break after each case.

Level 3: Loops (Q16-23)

16. Write a program that prints the numbers 1 to 10, each on its own line, using a for loop.

Hint: for (int i = 1; i <= 10; i++) { ... }

17. Write a program that prints all even numbers from 1 to 50.

Hint: You can either check i % 2 == 0 inside the loop, or step by 2 in the loop update.

18. Write a program that calculates the sum of all numbers from 1 to N, where N is entered by the user.

Hint: Start a variable sum at 0 and add i to it on every loop iteration.

19. Write a program that calculates the factorial of a number entered by the user (factorial of 5 is 5*4*3*2*1).

Hint: Start a variable result at 1, then multiply it by i for i from 1 to n.

20. Write a program that reads numbers from the user one at a time until the user enters 0, then prints how many numbers were entered.

Hint: Use a while loop that keeps reading input while the value is not equal to 0.

21. Write a program that prints a multiplication table (1 to 10) for a number entered by the user.

*Hint: One for loop from 1 to 10, printing number * i each time.*

22. Write a program that checks whether a number entered by the user is a prime number.

Hint: Check if any number from 2 up to n-1 (or up to the square root of n) divides n evenly.

23. Write a program that reverses the digits of an integer entered by the user (for example 1234 becomes 4321), without using arrays or strings.

Hint: Repeatedly take the last digit with n % 10 and remove it with n = n / 10, building the reversed number as you go.

Level 4: Arrays (Q24-30)

24. Declare an array of 10 integers, fill it using a for loop where each element equals its index, and print all the elements.

Hint: int arr[10]; then arr[i] = i; inside a loop from 0 to 9.

25. Write a program that reads 5 integers from the user into an array and prints their sum and average.

Hint: Read into the array with a loop using scanf("%d", &arr[i]);

26. Write a program that finds the largest and smallest values in an array of numbers entered by the user.

Hint: Assume the first element is both the max and min, then update as you scan the rest.

27. Write a program that reverses the elements of an array in place (without using a second array), and prints the result.

Hint: Swap the first and last elements, then move inward, using a temp variable.

28. Write a program that counts how many times a specific number (entered by the user) appears in an array.

Hint: Loop through the array and increment a counter whenever arr[i] equals the target.

29. Write a program that sorts an array of numbers in ascending order using bubble sort.

Hint: Compare each pair of neighbouring elements and swap them if they are in the wrong order; repeat for the whole array multiple times.

30. Write a program that uses a 2D array (a 3x3 matrix) to store 9 numbers entered by the user, then prints the matrix and the sum of all elements.

Hint: int matrix[3][3]; use two nested for loops, one for rows and one for columns.

Level 5: Strings and Functions (Q31-38)

31. Write a program that reads a word from the user and prints its length, without using the strlen() library function.

Hint: Loop through the character array until you hit the null terminator '\0', counting characters.

32. Write a program that checks whether a word entered by the user is a palindrome (reads the same forwards and backwards, e.g. "level").

Hint: Compare characters from the front and back of the string moving toward the middle.

33. Write a program with a function called square(int n) that returns n squared. Call it from main() for a number entered by the user.

*Hint: A function needs a return type, a name, parameters, and a return statement: int square(int n) { return n*n; }*

34. Write a function called isEven(int n) that returns 1 if n is even and 0 otherwise. Use it inside main() to test several numbers.

Hint: A function can return an int that acts like a boolean (1 for true, 0 for false).

35. Write a function called sumArray(int arr[], int size) that returns the sum of all elements in the array. Call it from main().

Hint: When passing an array to a function, you also need to pass its size separately.

36. Write a function that converts a lowercase letter to uppercase without using any library function, then use it to convert a whole word entered by the user.

Hint: The relationship between 'a' and 'A' in ASCII is a fixed number — subtract that difference.

37. Write a program with a function called power(int base, int exp) that calculates base raised to exp using a loop (not the pow() library function).

Hint: Multiply a result variable by base, exp number of times, starting from result = 1.

38. Write a recursive function called factorial(int n) that calculates factorial using recursion instead of a loop.

Hint: A recursive function calls itself with a smaller value; make sure to include a base case, e.g. factorial(0) = 1, or the recursion never stops.

Level 6: Pointers and Structures (Q39-45)

39. Declare an integer variable and a pointer to it. Print the variable's value, its address, and the value through the pointer.

*Hint: Use & to get an address and * to dereference a pointer: int *p = &x; printf("%d", *p);*

40. Write a function called swap(int *a, int *b) that swaps two integers using pointers, and call it from main() to swap two variables.

Hint: Pass the addresses of the variables when calling: swap(&x, &y);

41. Write a program that uses a pointer to traverse and print every element of an array, instead of using array indexing (arr[i]).

*Hint: A pointer can be incremented: *(ptr + i), or ptr++ moves it to the next array element.*

42. Define a structure called Student with members name (char array), age (int), and gpa (float). Create one Student variable, fill in its details, and print them.

Hint: struct Student { char name[50]; int age; float gpa; }; then access members with the dot operator.

43. Create an array of 3 Student structures (from the previous question), fill each one using a loop, and print all their details.

Hint: struct Student list[3]; access each one's members with list[i].name, list[i].age, etc.

44. Write a function that takes a pointer to a Student structure and prints that student's details.

*Hint: Use the arrow operator: void printStudent(struct Student *s) { printf(s->name); }*

45. Write a program that dynamically allocates memory for an array of N integers (N entered by the user) using malloc(), fills it with input values, prints them, and then frees the memory.

*Hint: int *arr = (int*) malloc(n * sizeof(int)); always call free(arr); when you are done with it.*

Level 7: Putting It All Together (Q46-50)

46. Write a program that reads a sentence (using fgets, not scanf) and counts how many words it contains.

Hint: Words are usually separated by spaces; count transitions from space to non-space, or simply count spaces and add 1.

47. Write a program that reads 10 numbers into an array and uses a function to sort them (bubble sort or similar), and another separate function to print the sorted array.

Hint: Split the work into two functions: void sortArray(int arr[], int size) and void printArray(int arr[], int size).

48. Write a program that implements a simple linked list with 3 manually created nodes, each storing an integer, and prints all the values by walking through the list.

*Hint: struct Node { int data; struct Node *next; }; link them by setting node1.next = &node2; etc.*

49. Write a program that writes 5 numbers entered by the user into a text file called numbers.txt, then reads them back from the file and prints their sum.

Hint: Use fopen("numbers.txt", "w") to write, then fopen("numbers.txt", "r") to read, and always fclose() when done.

50. Write a menu-driven program (using a loop and a switch statement) that lets the user repeatedly choose to add, view, or delete a number from an array, until they choose to exit.

Hint: This combines almost everything: loops, switch, arrays, and functions. Build it one menu option at a time and test each before adding the next.